



1 Introduction

This assignment is consisted of two major parts. In the first part, you will answer homework questions related to concepts in perception. In the second part, you will implement a camera-based lane detection module using a neural network. The module will take a stream of raw images as input, and it will produce a stream of annotated images with the lane area marked as the output. You will implement the functions detailed in later in the documentation. You will also write a brief report `mp1_<groupname>.pdf`. In the assignment, you will gain some experience with Robot Operating System (ROS2) [?], and you are encouraged to use Pytorch functions to help you get familiar with neural networks. This document will guide you to the first steps on implementing the lane detection module. You are free to take advantage of tutorials and documentations available online. **Make sure to cite all resources in your report. All the regulations for academic integrity and plagiarism spelled out in the [student code](#) apply.**

Learning objectives

- Binary Segmentation with a Neural Network
- Coordinate Transformation
- Working with ROS – Gazebo

System requirements

- Ubuntu 24.04
- ROS 2 Jazzy

2 HW1: Perception

Problem 1 [5 points] [Individual] An input matrix has spatial size $W \times H = 28 \times 28$ with $C_{\text{in}} = 1$ channel. A convolutional layer uses $C_{\text{out}} = 6$ filters, kernel size $k = 3$, stride $s = 2$, padding $p = 1$, and dilation $d = 2$.

- (a) [3 points] Write the general formula for the output spatial dimension as a function of W, k, s, p, d .
- (b) [1 points] Compute the numerical output shape (channels $\times$ width $\times$ height) for the given layer.
- (c) [1 points] Compute the effective kernel size $d(k - 1) + 1$ for $d = 1$ and for $d = 2$ (with $k = 3$ fixed).

Problem 2 [5 points] [Individual] A thin-lens camera has focal length $f = 35$ mm. An object of height 1.8 m stands at a distance $Z = 10$ m from the camera.

- (a) [2 points] Using the pinhole/thin-lens projection relation, compute the image height (in mm) on the sensor.
- (b) [2 points] Compute the magnification $m = (\text{image height})/(\text{object height})$.
- (c) [1 points] Compute the magnification if Z instead doubles to 20 m (with $f = 35$ mm fixed).

Problem 3 [15 points] [Individual] Consider a single neuron with

$$\hat{y} = f(wx + b),$$

where $f(z) = z^2$, the input $x = 2$, and the target $y = 8$. The initial parameters are $w = 1$ and $b = \frac{1}{2}$.

- (a) [5 points] Compute the forward pass and loss using the mean squared error:

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2.$$

- (b) [5 points] Compute the gradients $\partial\mathcal{L}/\partial w$ and $\partial\mathcal{L}/\partial b$.
- (c) [5 points] Update w and b using gradient descent with a learning rate $\alpha = 0.001$.

Problem 4 [Individual] (20 points) You are given the pixel coordinates $(u, v) = (400, 300)$ of a point observed by a camera, along with the camera's intrinsic matrix

$$\mathbf{K} = \begin{bmatrix} 800 & 0 & 320 \\ 0 & 800 & 240 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad [\mathbf{R} \mid \mathbf{t}] = \begin{bmatrix} 0.6 & -0.8 & 0 & 1 \\ 0.8 & 0.6 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix}.$$

Assuming the depth of the point in the camera coordinate system, z_c , is 5 m, calculate the 3D world coordinates x_w, y_w, z_w of the point.

Problem 5 [20 points] [Individual] Consider the scalar-valued network

$$g(v_3, v_2, v_1, z) = v_3 \sigma(v_2 \sigma(v_1 z)),$$

where $v_1, v_2, v_3, z \in \mathcal{R}$ and $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function. Let $x_1 = v_1 z$, $h_1 = \sigma(x_1)$, $x_2 = v_2 h_1$, $h_2 = \sigma(x_2)$, so that $g = v_3 h_2$.

- (a) [5 points] Derive $\partial\sigma/\partial x$ two ways: (1) in terms of x and e^{-x} only, and (2) in terms of $\sigma(x)$ only, with the target form $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ (credit given only if you show all steps).

- (b) [5 points] Compute $\partial g / \partial v_3$. Which forward-pass value should be retained to make this cheap?
- (c) [5 points] Compute $\partial g / \partial v_2$, using the form of σ' from part (a). Which values should be retained?
- (d) [5 points] Compute $\partial g / \partial v_1$, again using the form of σ' from part (a). Which values should be retained, and in what order should $\partial g / \partial v_3$, $\partial g / \partial v_2$, $\partial g / \partial v_1$ be computed (forward order or reverse order of the forward pass)?

Problem 6 [15 points] [Individual] A camera is calibrated using known 3D points $(x_w^{(i)}, y_w^{(i)}, z_w^{(i)})$ matched to captured pixels $(u^{(i)}, v^{(i)})$. The unknown 3×4 projection matrix P (entries $p_{11}, \dots, p_{34}$) satisfies

$$u^{(i)} = \frac{p_{11}x_w^{(i)} + p_{12}y_w^{(i)} + p_{13}z_w^{(i)} + p_{14}}{p_{31}x_w^{(i)} + p_{32}y_w^{(i)} + p_{33}z_w^{(i)} + p_{34}}$$

- (a) [5 points] Rearrange the $u^{(i)}$ equation into the linear form $a^{(i)\top} p = 0$, where $p = [p_{11}, \dots, p_{34}]^\top$. Write out $a^{(i)}$ explicitly.
- (b) [5 points] For the calibration point $(x_w, y_w, z_w) = (2, 1, 3)$ observed at pixel $u = 150$, write the numeric coefficient row from part (a).
- (c) [5 points] Stacking rows from many points gives $Ap = 0$. Since p is defined only up to scale, we solve $\min_p \|Ap\|^2$ subject to $\|p\|^2 = 1$. What is p , in terms of $A^\top A$?

Problem 7 [20 points] [Individual] A stereo rig has two cameras with identical intrinsics $f_x = f_y = f = 600$, $o_x = 320$, $o_y = 240$. Unlike a standard rig, the right camera is offset from the left by *both* a horizontal baseline $b_x = 0.3$ m and a small vertical baseline $b_y = 0.05$ m (e.g., due to imperfect mounting). A 3D point (x, y, z) , expressed in the left camera's frame, projects to

$$u_l = f \frac{x}{z} + o_x, \quad v_l = f \frac{y}{z} + o_y, \quad u_r = f \frac{x - b_x}{z} + o_x, \quad v_r = f \frac{y - b_y}{z} + o_y$$

- (a) [8 points] Derive two *separate* expressions for the depth z : one using only the horizontal disparity $u_l - u_r$ and b_x , and one using only the vertical disparity $v_l - v_r$ and b_y . (These should agree only if the rig's vertical alignment is accurate; this gives a way to sanity-check calibration.)
- (b) [6 points] The point is observed at $(u_l, v_l) = (370, 270)$ and $(u_r, v_r) = (340, 265)$. Compute z using *both* formulas from part (a), and confirm they agree.
- (c) [6 points] Using the value of z from part (b), solve for x and y .

3 MP1 Overview

Problem. Lane keeping is an important driver assistance system in modern vehicles that is enabled by powerful sensors and algorithms. This MP will help you develop the perception component of lane keeping using a single camera and a neural network in a simulated environment. What you learn and build here will be useful later for other assignments and projects.

3.1 System Diagram

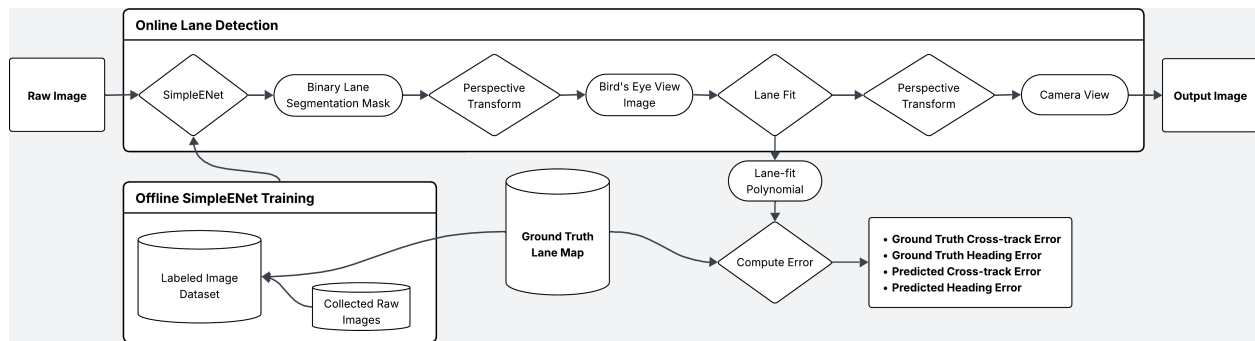


Figure 1: Lane Detection System Diagram

The system diagram of our lane detection pipeline is shown in Figure 1. First, you will train a simple neural network of your design using the ground-truth lane map to generate labels for collected raw images. During lane detection, the trained model outputs a binary mask image of lane lines given a raw image. This segmentation mask then gets converted to a bird's eye view (BEV) image. Finally, a lane fitting module fits polynomials to the lanes, returning an image with the detected lanes overlaid to the raw image as well as the polynomials. The system's performance is evaluated by computing errors, such as cross-track and heading errors, by comparing the fitted lane polynomial with the ground truth lane map. This MP requires you to implement only some of the said components, as listed under Section 3.2. Of course, you are encouraged to experiment with the entire codebase, as long as those changes are reverted upon submission.

3.2 Implementation Details

Network Architecture Design an *architecture* of a simple neural network for lane segmentation.

Neural Network Training Implement the *training loop* for a simple neural network.

Lane Segmentation Run *inference* on the neural network to obtain a binary lane segmentation image.

World-to-pixel Coordinate Transformation Perform a *world-to-pixel coordinate transformation* to obtain an accurate bird's eye view (BEV) image.

Lane Fitting Using the *sliding windows (histogram) technique*, fit the lanes using two second-order polynomials that represent each lane line, respectively.

Error Metrics Calculate cross-track and heading error which can be used as an input to a lane keeping assist controller.

All the scripts you will need to modify are:

- `code/src/mp1/scripts/simple_enet.py`
- `code/src/mp1/scripts/train.py`
- `code/src/mp1/scripts/model_utils.py`
- `code/src/mp1/scripts/generate_bev_config.py`
- `code/src/mp1/scripts/lane_detect.py`

3.2.1 Installation & Launching

The following commands assume you are using the lab computers. If you wish to setup a WFH solution seek help from a TA or your peers. WFH solutions won't guarantee support by staff if you run into issues.

```
git clone git@github.com:safeautonomy-illinois-students/fa26-mp1-release.git
cd fa26-mp1-release/code
colcon build --symlink-install
source install/setup.bash # NOTE: when launching, make sure to source this
python3 -m venv .venv
source .venv/bin/activate # NOTE: activate for each new terminal
pip install -r src/mp1/requirements.txt
```

If everything built and installed correctly, you should be ready to launch Gazebo, Rviz, and the simulated GEM vehicle.

```
# from any directory assuming you sourced the correct setup.bash file
# it will take a few seconds for everything to start up
ros2 launch mp1 mp1_launch.launch
```

3.2.2 Network Architecture

In this section, we will implement a neural network capable of image segmentation to determine lanes within an image. The model takes a gray-scale image as input and produces pixel-wise predictions, classifying each pixel as either part of a lane or not. We will build a simplified version of ENet, a lightweight and efficient network used for real-time segmentation tasks. We provide four modules to build Simple ENet, and your task is to complete the missing blanks in the InitialBlock. The detailed flow of the InitialBlock is provided below:

- InitialBlock takes the input image with the shape of (1, 384, 640)
- Input image $\rightarrow$ Convolution Layer $\rightarrow$ Feature f_1 with the shape of (out_channels-1, 192, 320)
- Input Image $\rightarrow$ Maxpooling Layer $\rightarrow$ Feature f_2 with the shape of (1, 192, 320)
- Concatenate f_1 and f_2 along the second dimension (axis=1) $\rightarrow$ Batch Norm $\rightarrow$ Activation $\rightarrow$ Output with the shape of (out_channels, 192, 320)

To begin training, you must complete this module in `code/src/mp1/scripts/simple_enet.py` according to the description above.

After building the modules for Simple ENet, we need to obtain images to train the model on. To do this, we have provided a script where you can control the GEM vehicle in the Gazebo environment using WASD keys. In a new terminal (assuming you have already launched) run the following command:

```
# in code/src/mp1
python3 scripts/drive.py
```

Once you have collected a training set, we need to generate the ground truth image to train the model. Since we are in a simulated environment, generating these masks is trivial. In the real world, generating ground truth data is a much more difficult problem. We have provided a script that will convert your captured images into binary masks that segment the lane lines. Run the following command:

```
# in code/src/mp1
python3 scripts/preprocess.py
```

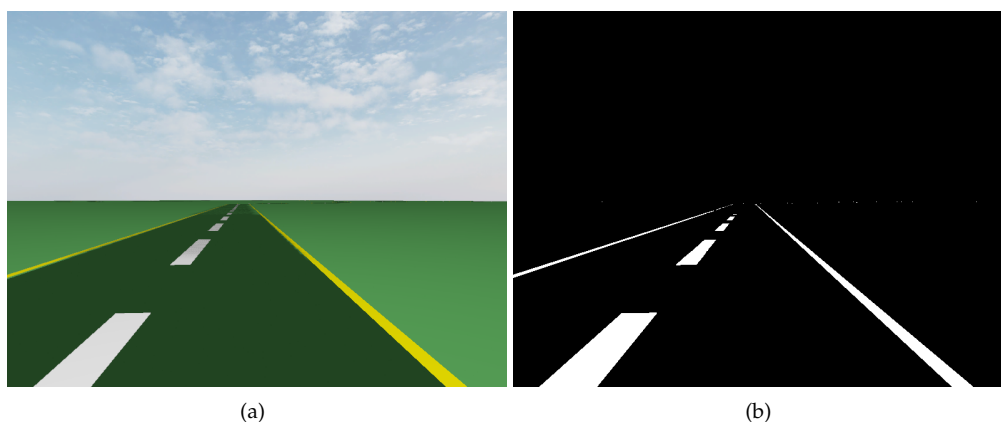


Figure 2: (a) Input raw image (b) Binary lane segmentation mask

3.2.3 Neural Network Training

In order to train the neural network, we need a training loop, which defines how the model processes data, computes loss, and updates its parameters through optimization. Your task is to choose the optimizer and update the model parameter with back propagation.

Optimizer is a key component of training, which adjusts the model's weights based on the computed gradients. You should select an appropriate optimizer (e.g., SGD, Adam, or AdamW) and initialize it with the model's parameters, along with essential hyperparameters such as the learning rate. During training, we iterate over the dataset using a DataLoader, moving input images and labels to the chosen computation device. After feeding the images into the model and performing a forward pass, we obtain the outputs and compute the loss. Then, we clear any previously accumulated gradients, perform a backward pass to compute gradients with respect to the loss, and use the optimizer to update the model parameters.

This completes one iteration of training. Over time, as the training progresses, the loss should gradually decrease and eventually converge, reflecting the model's improved ability to perform lane segmentation.

Once your Initial block is complete (see `code/src/mp1/scripts/simple_enet.py`), the training loop is complete (see `code/src/mp1/scripts/train.py`, and you have collected and processed your data (see above), you can begin training.

```
# in code/src/mp1
python3 scripts/train.py

# in a separate terminal, in code/src/mp1
# this should open a website, typically http://localhost:6006/
# here you can view training progress and sample images
python3 -m tensorboard.main --logdir=data/runs
```

3.2.4 Lane Segmentation

In this section, we will implement a binary lane segmentation pipeline using a neural network. The goal is to process a raw image and extract a binary mask highlighting lane pixels during the inference time.

The main steps are as follows:

1. **Resize** the input image to (640×384) , matching the model's expected input resolution.
2. **Grayscale and normalize** the image to single-channel float values in the $[0, 1]$ range.
3. **Convert** the preprocessed image into a PyTorch tensor and add the channel and batch dimensions to meet the model input requirements.
4. **Run inference** on the input using the pretrained model to get a pixel-wise classification output in real time.
5. **Post-process** the output by taking the argmax over the class dimension to get a predicted class map, convert it into a binary format, and finally resize it back to the original image dimensions.

The model is expected to output a two-channel tensor (background vs. lane), where the second channel corresponds to the lane class. By applying `argmax` over the channel axis, we extract the most probable class for each pixel. Multiplying this prediction by 255 gives us a binary mask with pixel values of either 0 (non-lane) or 255 (lane), suitable for visualization and downstream processing.

This function will serve as the backbone for visual lane perception in your module. Complete the functions in `code/src/mp1/scripts/model_utils.py`.

Once completed, you should get a `/lane_mask` topic when running:

```
# in code/src/mp1
python3 scripts/drive.py --run_model
```

You can view this topic in real-time through Rviz. Click Add (bottom left side) -> By topic -> `/lane_mask` -> Image. Also if your camera view is not appearing, Add -> By topic -> `/camera` -> `/image_raw` -> Image.

3.3 World-to-pixel Coordinate Transformation

In this section, you will implement world-to-pixel transformation to obtain an accurate bird's eye view (BEV) image. What is an "accurate" BEV image? A proper BEV should preserve *collinearity* (i.e., all points lying on a line initially still lie on a line after the transformation) as well as the resulting image displaying *parallel lane lines*. We achieve these by using a 3-by-3 perspective transformation matrix.

$$\begin{bmatrix} tu \\ tv \\ t \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (1)$$

Here (x, y) and (u, v) are the coordinates of the same point in the coordinate systems of the original perspective and the new perspective. To find the matrix, we need to find the location of 4 points on the original image, namely the *source points*, and map the corresponding 4 points, on the BEV. Any 3 of those 4 points should not be on the same line. Put those 2 groups of points into `cv2.getPerspectiveTransform()`, the output will be the transformation matrix. Pass the matrix into `cv2.warpPerspective()` and we will have the final BEV image.

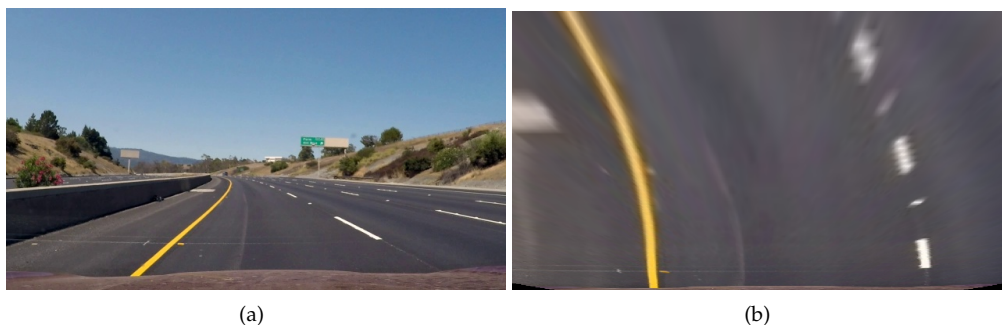


Figure 3: (a) Raw image (b) Bird's Eye View (BEV) image after transformation.

Instead of choosing the source points manually, we will perform a world-to-pixel coordinate transformation of the corners of the rectangle in the world frame, determined by its height, width and the distance between its lower edge and the base link of the vehicle, shown in Figure 4. This offers several key advantages:

1. The resulting BEV image displays perfectly parallel lane lines
2. It enables a quick iteration of many different BEV configurations without much trial-and-error.

You are free to experiment with the hyperparameters, but for the submission, please return them to the original values for a more reliable evaluation. Once you implement the TODO sections in `generate_bev_config.py`, save the BEV configuration as a JSON format, which other modules then can access, by running:

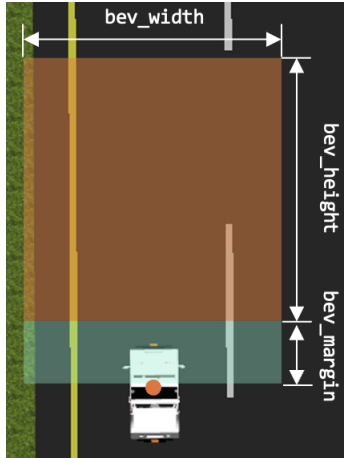


Figure 4: BEV rectangle with the hyperparameter names

```
# in code/src/mp1
python3 scripts/generate_bev_config.py
```

3.4 Error Metrics

Once the color camera image is converted to a binary lane-line image via the trained model and converted to a BEV, we can now fit lane lines to it. This algorithm is already provided for you. Your job is to calculate the projected cross-track error and heading error from the projected lane-lines. In `lane_detect.py`, complete the `compute_error` class member function with the help of the provided comments, polynomial coefficients of the centerline (defined in pixels), and the ground-truth XTE (meters) and HE (degrees).

Once complete, run `lane_detect.py` to be presented with the real-time annotated BEV as well as projected and ground-truth XTE and HE.

```
# in code/src/mp1
python3 scripts/lane_detect.py
```

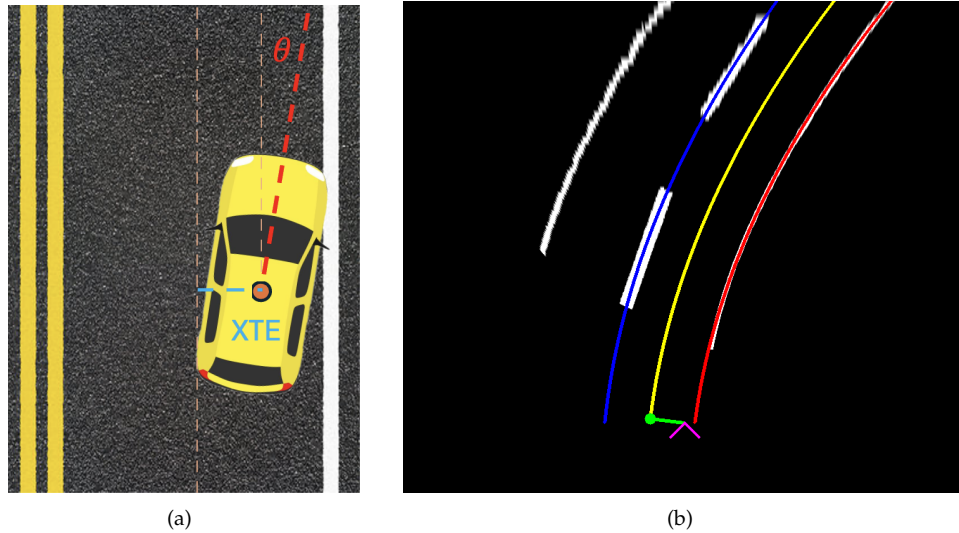


Figure 5: (a) concept image showcasing XTE and HE (b) Birds-Eye View of lane lines with lane detection and XTE metrics. White: lane-line detections, Blue: left lane-line, Red: right lane-line, Yellow: center-line, Green: XTE offset line, and Purple: camera origin

3.5 Report

Each group should upload a short report with following questions answered.

Problem 8 [Group] (15 points) How do you determine whether a model is overfitting to the training set, and what methods can be used to mitigate it? How do you diagnose underfitting, and what methods can be used to reduce it? Please list two methods, respectively.

Problem 9 [Group] (20 points) What problem might occur if a model is trained mainly on sunny-day images but tested on snowy-day images? List two methods that could improve the model's performance on snowy days. Briefly describe how each method works.

Problem 10 [Group] (30 points) Record a short videos of Rviz window and Gazebo to show that your code works. You can either use screen recording software or smart phone to record. Please ensure that your links to videos are functional before submission.

Problem 11 [Group] (25 points) We will grade your model on our private test set based on the accuracy of lane detection using the checkpoints you provide. Also, explain how you selected the BATCH_SIZE, Learning Rate (LR), and Number of Epochs when you tune the hyperparameters. Please ensure that you submit the final checkpoints and training loss curve for grading. Make sure to move the final checkpoint into the mp1 folder as the data folder is gitignored.

Demo (10 points) You should be able to demonstrate your network performing well on novel data (e.g.) driving around to various parts of the track. We will also assess your network on a private testing set. Cross-track error and heading error will also be assessed.

3.6 Submission instructions

All files you change must be uploaded to your team's github repo by the deadline to be considered for grading. Problems 8-11 can be done in groups of 3. Figures of **training loss curve** should be submitted in the report. Students need to take a short videos clearly showing that their code works and answer other questions. Your video must show that your implementation can detect lanes when the car is **moving**. The videos can be uploaded to a website (YouTube, Vimeo, or Google Drive with public access) and you need to include the link in the report. One member of each team will upload the report `mp1_<groupname>.pdf` to Canvas. Please include the names and netids of all the group members and cite any external resources you may have used in your solutions. Also, please upload a **single** final checkpoint to your github. Note: the data directory is gitignored by default so move your final checkpoint to the `mp1` directory.